

Product Specification: Nutritional Software

1. Overview The Nutritional Software is a mobile or web-based application designed for students in nursing and exercise science programs. The primary function is to record, analyse, and interpret a user's dietary intake and compare it against national or international Recommended Dietary Intakes (RDIs) or Nutritional Requirements. It also includes features to capture personal demographic and health data, estimate energy expenditure, and potentially support educational use cases.

2. Target Users

- Bachelor of Nursing students
- Bachelor of Exercise students

3. Key Features

3.1. User Profile Setup

- Input required:
 - Name
 - Age
 - Gender
 - Ethnicity
 - Body Weight (kg/lbs)
 - Height (cm/ft)
 - Medical Conditions (select from list or free text)
 - Activity Level (Sedentary, Lightly Active, Active, Very Active)
 - Dietary Preferences or Restrictions (e.g., vegetarian, gluten-free)
- Data used to tailor nutritional recommendations.

3.2. Food Intake Logging

- Searchable food database (API or local, such as the provided Food files)
- Ability to:
 - Select food items
 - Specify portion sizes (grams, mL, servings)
 - Enter time and date of consumption
 - Tag meals (breakfast, lunch, dinner, snack)
- Nutrient data (macronutrients and micronutrients) pulled from food database.
- Add custom foods or recipes (manual input of nutrient data if food is not found).

3.3. Nutritional Analysis Component

- Compare total intake to RDIs based on user profile.

- Breakdown of:
 - Energy (kcal/kJ)
 - Protein, Carbohydrates, Fats
 - Vitamins and Minerals
- Highlight deficiencies or excesses.
- Daily, weekly, and custom date range reports.

3.4. Energy Expenditure Estimation

- Methods:
 - Basal Metabolic Rate (BMR)
 - Activity level multiplier for Total Daily Energy Expenditure (TDEE)
 - Optional manual input of physical activity
- Optional fitness test result inputs (e.g., VO2 max, BMI, etc.)

3.5. Health and Fitness Inputs

- Capture results from:
 - Fitness assessments (e.g., beep test)
 - Health questionnaires (e.g., lifestyle survey)
- Store results in user profile for historical reference.

3.6. Educational Component

- Optional mode for teaching purposes:
 - Guided tutorials or simulations (e.g., case studies)
 - Explanations of nutrient functions, RDIs, etc.
 - Quizzes to reinforce learning
 - Scenario-based intake entry and analysis

3.7. Reports and Exporting

- Export options:
 - PDF or CSV reports of intake vs. RDIs
 - Summary for assignments or assessments
- Data visualisation:
 - Graphs and charts for trends over time
 - Pie charts for macronutrient distribution

4. Technical Specifications

- Platform: Web app (responsive) and/or Mobile app (iOS, Android)
- Backend:
 - Node.js or Python (Django/Flask) with RESTful API

- Database: sqlite, PostgreSQL or MongoDB
- Integration with food data APIs (files provided, could explore other APIs)
- Frontend:
 - React (Web), React Native (Mobile), Svelte, Vanilla js
 - Data visualisation: Chart.js

5. UX/UI Considerations

- Simple, intuitive UI for food logging
- Clear data entry forms for personal and health data
- Dashboard view summarising key metrics
- Accessibility and responsiveness
- Support light and dark mode for usability

6. Security and Privacy

- User authentication (OAuth, email/password)
- Data encryption (at rest and in transit)
- Compliant with relevant data protection regulations (e.g., GDPR)

7. Testing Requirements

Test strategy

Scope and objectives

Testing types (e.g. unit testing, integration, regression, acceptance)

Automation: e.g. selenium for UI, xUnit for unit

Environments: Dev, staging

Test artifacts

Test cases (inputs, expected outputs)

Test scripts (e.g., automated test code)

Test data (input sets, boundary conditions)

Test coverage reports

Manual tests need to be documented

- Unit and integration testing for:
 - Food database search
 - Nutritional calculations
 - Energy expenditure estimation
- Usability testing
- UI testing on different screen sizes
- Accessibility testing (WCAG compliance)

8. Development Milestones The project will be executed using agile methodology over **8 weeks of development**, divided into **4 sprints (each 2 weeks)**. Development will follow an **iterative and incremental approach**, enabling the team to deliver functional components in each sprint, validate assumptions early, and adapt based on feedback.

- **Sprint 1: Foundation & Core Flow (Vertical Slice)**
 - Define core requirements and wireframes
 - Set up backend and frontend environments
 - Implement user profile setup (basic version)
 - Implement food logging UI with mock data
 - Display simple nutritional feedback using hardcoded thresholds
- **Sprint 2: Input Expansion & Basic Analysis**
 - Integrate live food database (API or local)
 - Expand user profile fields (e.g., medical conditions, preferences)
 - Implement accurate nutritional analysis engine (calculated from real data)
 - Begin work on daily report display
- **Sprint 3: Enhanced Analysis & Reporting**
 - Add energy expenditure estimation module
 - Complete full intake vs. RDI analysis
 - Add exportable reports (PDF/CSV)
 - Add charts/graphs for data visualisation
- **Sprint 4: Educational Features & Finalisation**
 - Build educational mode (tutorials, quizzes, scenarios)
 - Conduct comprehensive testing (unit, integration, accessibility)
 - Bug fixing, usability improvements
 - Prepare for final demo and handoff

10. Standup/retrospective guidelines

Retrospective

What went well

What did not go as well

What will we do differently

Standup format

What did I do since last stand-up

What will I do in this session

Were there any issues or blockers?

11. Software Inspections To support software quality and learning outcomes, formal inspections will be conducted at key points in the development cycle. Each inspection will involve peer review of artefacts for correctness, completeness, clarity, and maintainability.

Each inspection will involve a checklist-based review process with peer feedback, follow-up fixes, and summary documentation of findings and actions taken.

12. Software Inspections (requirements for assessment 3)

As part of software engineering best practices, you will conduct software inspections of your team's artefacts throughout the development process. These inspections will focus on identifying defects, assessing design and implementation quality, and ensuring maintainability and clarity.

You are required to:

1. Submit a reflection and checklist for each inspection session.
2. Evaluate artefacts, selected from:
 - Requirements / Wireframes
 - Code (functionality, robustness, code quality, logic, functions, modules)
 - Tests (test coverage, documentation)
 - Reports / Visualisation Output
 - UI/UX flows
3. Use the inspection checklists provided to guide your evaluation.
4. Submit:
 - Completed checklists
 - Summary of identified issues
 - Brief reflection on the impact of the inspection

Sample Inspection Checklists

For Requirements / Wireframes:

- Are functional and non-functional requirements clearly stated?
- Are all user roles and use cases covered?
- Is the UI layout consistent with user expectations and accessibility standards?
- Are all inputs and outputs for each screen accounted for?

For Code / Logic (e.g., Nutritional Analysis, Food Logging):

- Are function and variable names meaningful and consistent?
- Are calculations (e.g., energy, nutrients, RDIs) accurate?
- Are all edge cases handled (e.g., missing data, extreme values)?
- Is the logic modular and reusable?
- Are comments and documentation clear?

For Reports and Visualisation:

- Are reports correctly generated and formatted (PDF/CSV)?
- Do visual elements reflect the correct data values?
- Are chart axes and labels accurate and understandable?
- Are layout and typography accessible (contrast, size, screen reader support)?

For UI/UX Review:

- Is the interface intuitive and easy to navigate?
- Are components responsive across screen sizes?
- Are user flows logical and complete?
- Are error messages informative and actionable?
- Is there adequate feedback for user actions (e.g., saving, logging data)?
- Accessibility (e.g. WCAG)